System Architecture and Integration

TECHNICAL DOCUMENTATION

EventHub: School Event Registration System

Containerized with Docker Compose
and deployed by a Jenkins CI/CD pipeline

Group: Barazon Group

Course / Section: ITE 304

Instructor: Harwin Mendoza

Group Members:

Barazon, Angelo P.

Laxamana, Karl Kristian Kenneth N.

Villanueva, Martin M.

Tanglao, Joshua Lawrance S.

Table of Contents

1. Title Page, Group and Members 3

1.1 Members and roles 3

2. Project Description and Modules 3

2.1 Modules and responsibilities 4

2.2 API summary (catalog-api, reached through /api/) 4

2.3 Data model 4

3. Architecture 5

3.1 Containers and ports 5

3.2 Network and volumes 5

3.3 Request flow 6

4. Technology Stack 6

5. Dockerfile Explanations 7

5.1 catalog-api and report-service (three-stage build) 7

5.2 frontend and admin-panel (two-stage build) 8

5.3 proxy, db and Jenkins images 8

5.4 Security choices 8

6. Docker Compose Explanation 8

6.1 Dependencies and health checks 9

6.2 Environment variables 9

7. Jenkins Setup 10

7.1 Installation 10

7.2 Credentials 10

7.3 Job and trigger configuration 10

8. Pipeline Explanation 11

9. Testing Strategy 12

10. Rollback Procedure and Data Persistence Proof 13

10.1 How rollback works 13

10.2 Demonstration 13

10.3 Data persistence proof 15

11. Problems Encountered and Solutions 16

12. Individual Contributions and References 17

12.1 Individual contributions 17

Appendix A: Project Proposal 19

1. Title Page, Group and Members

Group name	Barazon Group
Course / section	ITE 304
Instructor	Harwin Mendoza
Submission date	12/10/2026
Source repository	https://github.com/apBarazon/EventHub

1.1 Members and roles

Member	Role	Main responsibilities
Barazon, Angelo P.	Project Lead / Scrum Master Backend and Database Engineer DevOps / CI-CD Support Infrastructure Support	Project Lead / Scrum Master: plans tasks and the timeline, runs group meetings, tracks progress, manages the Git repository (branches, merge requests) and integrates the final demo flow. Backend and Database Engineer: builds the API and service modules, designs the database schema, writes the seed and initialization scripts and writes the automated tests for the backend modules. DevOps / CI-CD Support: supports the Jenkins setup, the Jenkinsfile, the GitHub webhook and polling triggers, the email notification and the rollback parameter. Infrastructure Support: supports the Dockerfiles, docker-compose.yml, the reverse proxy, networks, volumes and health checks.
Laxamana, Karl Kristian Kenneth N.	DevOps / CI-CD Engineer	Sets up Jenkins in Docker, installs plugins, configures credentials and triggers, writes and maintains the Jenkinsfile.
Villanueva, Martin M.	Infrastructure Engineer	Designs the architecture, writes Dockerfiles and docker-compose.yml, configures the web server / reverse proxy, networks, volumes, and health checks.
Tanglao, Joshua Lawrance S.	Frontend, QA, and Documentation Lead	Builds the user-facing module, writes smoke tests, maintains the README and technical documentation, prepares slides.

2. Project Description and Modules

EventHub is a containerized web application for managing school event registration. Students browse upcoming events, create an account and submit a short registration form. An administrator creates and edits events, approves or rejects registrations and reads a summary report. When a registration is approved or rejected the student receives an in-app notification. The system runs as six containers behind one reverse proxy, and a Jenkins pipeline builds, tests, deploys and verifies it every time code is pushed to GitHub.

Implemented features: student sign-up and login; browsing events; registering for an event; a "My Event Registrations" view; admin event management (create, edit, delete); admin approval or rejection of registrations;

a rule that blocks approval once an event is full; notifications; and an admin report (totals, fill rate per event, full events, most popular event).

2.1 Modules and responsibilities

Module	Technology	Responsibility	Port
proxy	nginx 1.27-alpine	Single public entry point. Routes by URL path: `/`, `/admin/`, `/api/`, `/api/reports/`.	80 (published as 8090)
frontend	HTML, CSS, vanilla JavaScript on nginx-unprivileged	Student site: event cards, sign-up and login, registration form, My Event Registrations, notifications. The footer shows the build number.	8080
admin-panel	HTML, CSS, vanilla JavaScript on nginx-unprivileged	Admin site: dashboard and report, event create/edit/delete, approve or reject registrations, notifications.	8080
catalog-api	Node.js 20 (built-in `http`, `mysql2`)	Authentication, events, registrations, approval logic (full-event rule), notifications. Creates the admin account at startup. `GET /health`.	3000
report-service	Node.js 20	`GET /reports/summary` (admin only): totals, fill rate, full events, most popular event. Verifies the same signed token as catalog-api.	4000
db	MySQL 8.4	Tables `users`, `events`, `registrations`, `notifications`. Seeded with five sample events. Data stored on volume `db-data`.	3306 (not published)

2.2 API summary (catalog-api, reached through /api/)

Access	Endpoints
Public	`GET /health`, `GET /events`, `GET /events/:id`, `POST /auth/register`, `POST /auth/login`
Student (token)	`GET /auth/me`, `POST /events/:id/register`, `GET /registrations/mine`, `GET /notifications`, `POST /notifications/read-all`
Admin (token, role admin)	`POST /events`, `PUT /events/:id`, `DELETE /events/:id`, `GET /admin/registrations`, `POST /admin/registrations/:id/approve`, `POST /admin/registrations/:id/reject`; and in report-service `GET /reports/summary`

2.3 Data model

The schema in `db/init.sql` has four tables. `users` stores a salted password hash and a `role` (user or admin). `events` stores title, description, date, location, image URL and `max_participants`. `registrations` links a user to an event with the form fields and a `status` (pending, approved, rejected); a unique key on (user, event) allows one registration per student per event, and foreign keys cascade on delete. `notifications` stores messages for users.

The init script runs only when the database volume is empty.

3. Architecture

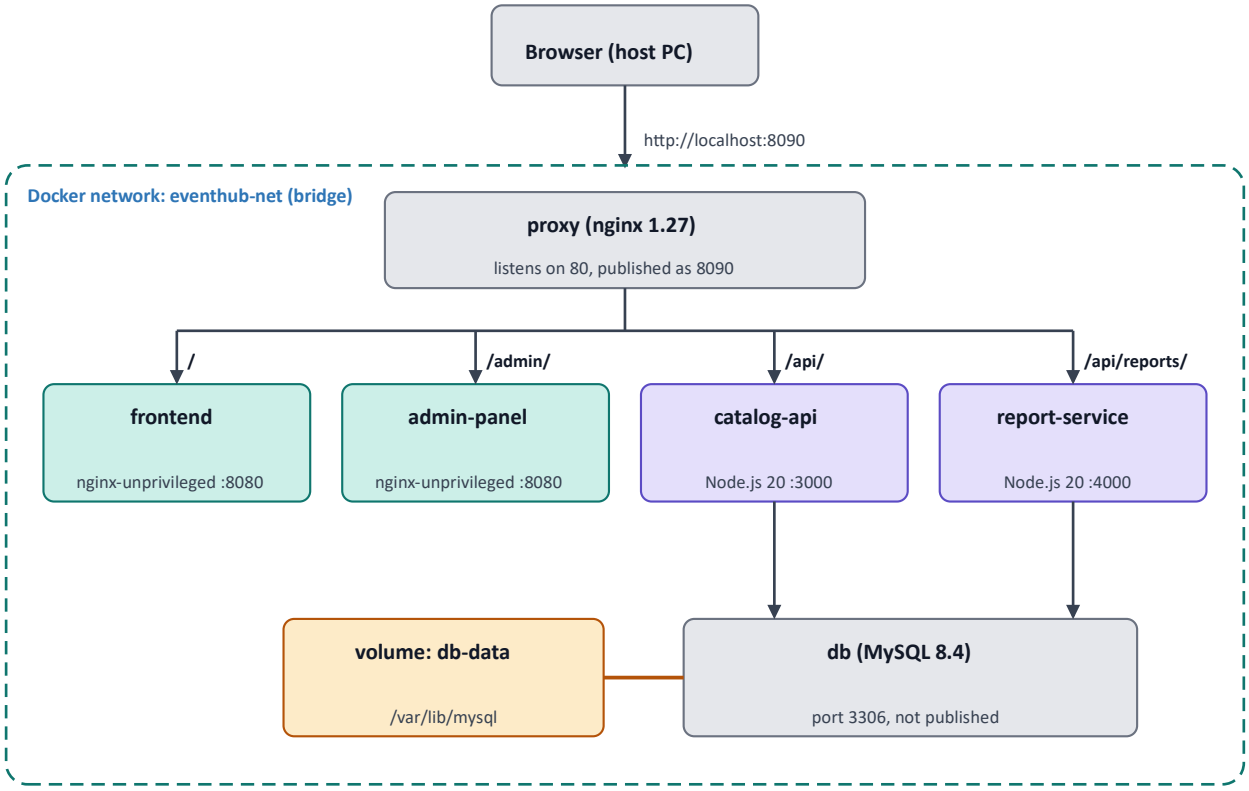


Figure 1. EventHub runtime architecture: containers, ports, network, volume and request routing.

3.1 Containers and ports

Container	Base image	Listens on	Published to host	Reachable from
proxy	nginx:1.27-alpine	80	8090 -> 80	Browser, all containers
frontend	nginxinc/nginx-unprivileged:1.27-alpine	8080	no	proxy only
admin-panel	nginxinc/nginx-unprivileged:1.27-alpine	8080	no	proxy only
catalog-api	node:20-alpine	3000	no	proxy only
report-service	node:20-alpine	4000	no	proxy only
db	mysql:8.4	3306	no	catalog-api, report-service

Only the proxy publishes a port. The host mapping is `8090:80`: the left number is the port on the Windows host, the right number is the port inside the proxy container. Internal ports (8080, 3000, 4000, 3306) are only reachable inside the Docker network, which keeps the services and the database off the public surface. Jenkins is not part of this stack: its web interface is published separately on port 8080 by its own Compose file (Section 7), which is why the application is published on 8090.

3.2 Network and volumes

- **Network.** One user-defined bridge network, `eventhub-net`. Containers resolve each other by service name (for example `catalog-api:3000` or `DB_HOST=db`).

- **Volume.** The named volume `db-data` is mounted at `/var/lib/mysql`. Because it lives outside the container, the data survives container re-creation, redeployments and rollbacks. It is deleted only by `docker compose down -v`.
- **Jenkins.** Jenkins runs from a separate Compose file with its own volume `jenkins_home` and publishes its web interface on port 8080 (see Section 7).

3.3 Request flow

Example: an administrator opens the report page.

1. The browser requests `http://localhost:8090/admin/`. The proxy matches `/admin/` and forwards to `admin-panel:8080`, which returns static files.
2. The page's JavaScript runs in the browser and calls `/api/auth/login`. The proxy strips `/api/` and forwards `/auth/login` to `catalog-api:3000`, which checks the password hash in MySQL and returns a signed token.
3. The page requests `/api/reports/summary` with the header `Authorization: Bearer ...`. The proxy forwards `/reports/summary` to `report-service:4000`.
4. report-service verifies the token with the shared `JWT_SECRET`, requires `role = admin`, runs its SQL query against the same database and returns the numbers.

The nginx rule is longest-prefix: `/api/reports/` is more specific than `/api/`, so report requests never reach catalog-api. catalog-api and report-service do not call each other; they are linked only by the shared database and the shared secret.

4. Technology Stack

Technology	Version	Used for	Justification
Docker, Docker Compose	Docker Desktop (Windows), Compose v2	Packaging and orchestration	Identical environment on every machine. One file describes six services, the network and the volume; images can be tagged per build, which makes rollback possible.
nginx	1.27 (alpine)	Reverse proxy and static file servers	Small and fast; path-based routing gives a single public port. The unprivileged variant serves static sites as a non-root user.
Node.js	20 (alpine)	catalog-api, report-service	One language across services; its built-in `http`, `crypto` and test runner keep the project to a single npm dependency, which means small images, fast builds and little supply-chain risk.
mysql2	3.11.3	MySQL driver (only dependency)	Promise API and parameterized queries, which protect against SQL injection.
MySQL	8.4	Database	MySQL is the only database our course has taught us, so using it lets us apply what we have learned and use it properly, instead of learning an unfamiliar database while building the infrastructure. It also fits the data: relational tables with foreign keys, enums and a unique constraint (one registration per student per event). The official image has an init-script hook.
HTML, CSS, vanilla JS	-	frontend, admin-panel	We have studied React but are not yet confident using it, and for a system this small it would add extra code, a build step and npm libraries that complicate the pipeline. Plain HTML, CSS and JavaScript are familiar to the whole team and need no build tool: the image just copies the files and sed stamps the build number.

Technology	Version	Used for	Justification
Jenkins	LTS, JDK 17	CI/CD	Pipeline as code (Jenkinsfile), GitHub webhook and SCM polling, credential store, parameterized builds, readable Stage View.
Git and GitHub	-	Version control	Jenkins pulls from the repository; commits give an audit trail; `git revert` undoes a bad change safely.
node:test	built into Node 20	Unit tests	No extra dependency; runs inside `docker build` so tests need no database or network.
Alpine Linux	3.20, node-alpine	Base images	Small images with a minimal attack surface; BusyBox `wget` is available for health checks.
scrypt and HMAC-SHA256	Node `crypto`	Passwords and login tokens	Salted password hashing, timing-safe comparisons and expiring signed tokens without extra libraries.

5. Dockerfile Explanations

5.1 catalog-api and report-service (three-stage build)

```
FROM node:20-alpine AS base            # shared layers
WORKDIR /app
COPY package*.json ./
RUN npm install --no-audit --no-fund    # all deps, cached until package files change
COPY src ./src
COPY tests ./tests
FROM base AS test                       # Jenkins: docker build --target test
RUN npm test                            # non-zero exit code fails the build
FROM node:20-alpine AS production       # fresh, minimal final image
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm install --omit=dev --no-audit --no-fund && npm cache clean --force
COPY src ./src                          # no tests in the shipped image
EXPOSE 3000
HEALTHCHECK --interval=15s --timeout=3s --start-period=20s CMD wget -qO-
http://localhost:3000/health || exit 1
USER node                               # run as non-root
CMD ["node", "src/server.js"]
```

- **Layers and caching.** `package*.json` is copied before the source, so the dependency layer is rebuilt only when dependencies change.
- **Multi-stage.** The `test` stage exists only for verification and is never deployed. The `production` stage starts again from a clean base and installs only runtime dependencies, so the shipped image contains no tests and no dev packages.
- **Security.** The process runs as the unprivileged `node` user. `.dockerignore` excludes `node_modules`, `.env` and `.git`, so secrets never enter the image. The report-service Dockerfile has the same structure with port 4000.

5.2 frontend and admin-panel (two-stage build)

```
FROM alpine:3.20 AS build
ARG BUILD_NUMBER=dev
WORKDIR /site
COPY src/ ./
RUN sed -i "s/__BUILD__/${BUILD_NUMBER}/g" *.html     # stamp the build number
FROM nginxinc/nginx-unprivileged:1.27-alpine
COPY --from=build /site /usr/share/nginx/html
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s CMD wget -qO- http://127.0.0.1:8080/ >/dev/null ||
exit 1
```

The page footer contains the placeholder `__BUILD__`. During the image build, Compose passes `TAG` (the Jenkins build number) as `BUILD_NUMBER`, and `sed` writes it into the HTML. The number is therefore part of the image, which is visible proof of which version is live and which version a rollback restored. The final image uses the non-root nginx variant listening on 8080. The health check uses `127.0.0.1` instead of `localhost` (see Section 11).

5.3 proxy, db and Jenkins images

- **proxy:** `nginx:1.27-alpine` plus one `COPY` of `nginx.conf` into `conf.d/default.conf`. Two layers, no custom code.
- **db:** `mysql:8.4` plus `init.sql` copied into `/docker-entrypoint-initdb.d/`. MySQL runs it once, when the data directory is empty.
- **Jenkins:** `jenkins/jenkins:lts-jdk17`. As root, it installs `ca-certificates curl gnupg`, adds Docker's signed apt repository (the key is fetched with `curl`) and installs `docker-ce-cli` and `docker-compose-plugin`, then switches back to the `jenkins` user. Only the Docker client is inside the image; the daemon is the host's.

5.4 Security choices

- Non-root runtime users: `node` for the APIs and the unprivileged nginx variant for static sites.
- Small Alpine base images with pinned tags; production images contain no tests or dev dependencies.
- No secrets in images or Git: `.env` is git-ignored and docker-ignored, values are injected at runtime, and Jenkins stores the real file as a credential.
- Only the proxy is published; the database and internal services are not reachable from the host or the network.
- Passwords are stored as salted scrypt hashes; tokens are HMAC-signed with an expiry (8 hours) and compared in constant time.
- Known trade-off: the Jenkins container runs as root with the host's Docker socket mounted. This is acceptable for a local lab, but it gives Jenkins full control of the host's Docker and would need a safer design in production.

6. Docker Compose Explanation

The root `docker-compose.yml` defines six services, one network and one volume. Every service has `restart: unless-stopped` and joins `eventhub-net`. Each image name is `eventhub/<service>:${TAG:-latest}`: Jenkins sets `TAG` to the build number, and the same variable selects the version to run during a rollback.

6.1 Dependencies and health checks

Service	depends_on	Health check	Timing
db	-	`mysqladmin ping` as root	every 10 s, timeout 5 s, 10 retries, 30 s start period
catalog-api	db (service_healthy)	`wget localhost:3000/health` (runs `SELECT 1`)	every 15 s, timeout 3 s, 20 s start period
report-service	db (service_healthy)	`wget localhost:4000/health` (runs `SELECT 1`)	every 15 s, timeout 3 s, 20 s start period
frontend, admin-panel	-	`wget 127.0.0.1:8080/`	every 15 s, timeout 3 s
proxy	frontend, admin-panel, catalog-api, report-service (start order)	none in the Dockerfile or Compose file	verified by the pipeline smoke test

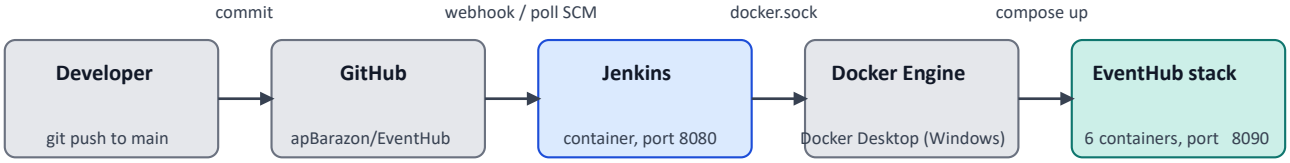
The APIs wait for a *healthy* database, not merely a started one, so they do not try to connect while MySQL is still initializing. Note that `depends_on` is applied by `docker compose up` only; when Docker Desktop restarts containers by itself after a reboot, there is no ordering (see Section 11, problem 4).

6.2 Environment variables

Variable	Used by	Purpose
`DB_ROOT_PASSWORD`	db	MySQL root password (`MYSQL_ROOT_PASSWORD`); also used by the DB health check.
`DB_USER`, `DB_PASSWORD`	db, catalog-api, report-service	Application database account created by MySQL and used by both APIs.
`DB_HOST`, `DB_NAME`	catalog-api, report-service	Fixed in Compose: `db` (the service name) and `eventhub`.
`JWT_SECRET`	catalog-api, report-service	Key used to sign and verify login tokens; must be the same in both.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	catalog-api	Admin account created at first start (if the email does not exist yet).
`TAG`	Compose, Jenkins	Image tag and build number; defaults to `latest` (images) and `dev` (build stamp).

Values come from a `.env` file that is excluded from Git. `.env.example` documents the names. In Jenkins the real file is stored as a secret-file credential and copied into the workspace only while a build runs. The admin account is created by application code (`ensureAdmin()`), not by `init.sql`, so the admin password never appears in the repository.

7. Jenkins Setup



Jenkins runs docker build / docker compose through the mounted Docker socket of the host.

Figure 2. CI/CD overview: GitHub triggers Jenkins (webhook, with polling as fallback), which drives the host's Docker Engine.

7.1 Installation

Jenkins runs as a container defined in `infra/jenkins/docker-compose.yml` on Windows with Docker Desktop:

```
services:
  jenkins:
    build: .
    container_name: jenkins
    user: root          # classroom/lab only: lets Jenkins use the Docker socket
    restart: unless-stopped
    ports:
      - "8080:8080"     # Jenkins web UI at http://localhost:8080
      - "50001:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock

volumes:
  jenkins_home:
```

1. Start it with `docker compose up -d --build` in `infra/jenkins` and open `http://localhost:8080`.
2. Unlock Jenkins with the initial admin password printed by `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`.
3. Install the suggested plugins, which provide Git, Pipeline and Credentials Binding. The GitHub plugin supports the webhook trigger (githubPush) and the Mailer plugin supports the email notification (mail step). Then create the admin user.
4. 4. Configure the email settings under Manage Jenkins > System: the SMTP server and port, and an app password for the sending account. The password is stored in Jenkins, not in the repository.

7.2 Credentials

One credential of kind **Secret file** with ID `eventhub-env` holds the real `.env` (database, JWT and admin settings). The pipeline binds it to a variable, copies it to `.env` in the workspace for Build and Deploy, and deletes it in the post section. Because the repository is public, no GitHub credential was needed.

7.3 Job and trigger configuration

Setting	Value
Job type / name	Pipeline, named `eventhub`
Definition	Pipeline script from SCM (Git), repository `https://github.com/apBarazon/EventHub`, branch `*/main`, script path `Jenkinsfile`

Setting	Value
Trigger	`pollSCM('H/2 * * * *')` declared in the Jenkinsfile: Jenkins checks GitHub about every two minutes and builds when there is a new commit. `githubPush()` is declared next to it: a GitHub webhook (payload URL /github-webhook/) starts the build within seconds of a push, and polling remains as the fallback.
Build parameter	`ROLLBACK_TAG` (string, empty by default). Empty means a normal build; a build number triggers a rollback (Section 10).
Options	`disableConcurrentBuilds()` (no overlapping deploys) and `buildDiscarder` keeping the last 15 builds.

Why both a webhook and polling? A GitHub webhook reacts within seconds, but it needs a URL that GitHub can reach, and Jenkins here runs on localhost, so it works only while the machine is exposed through a tunnel. Polling needs no public URL, so it stays as the fallback and the pipeline still starts by itself when the tunnel is off. Jenkins registers the triggers from the Jenkinsfile only after the job has run once, so the first build was started manually.

8. Pipeline Explanation

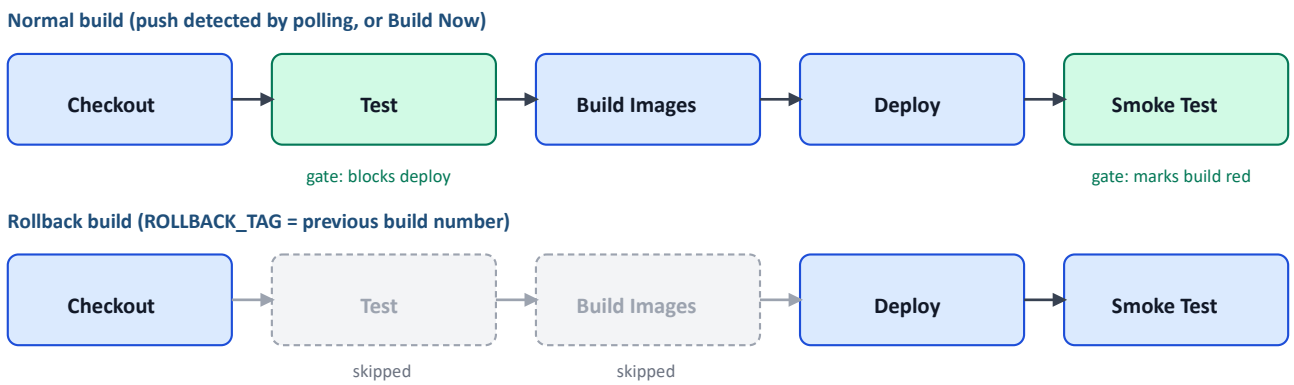


Figure 3. Pipeline stages. Normal builds run all five; a rollback build skips Test and Build Images.

Stage	What it does	What happens on failure
Checkout	Clones the repository at the commit that triggered the build (`checkout scm`).	Build fails immediately; nothing is built or deployed, the running site is untouched.
Test	Runs `docker build --target test` for catalog-api and report-service; each runs `npm test` inside the build.	A failing test fails the image build, the stage and the build. Build Images and Deploy never run, so the live version stays unchanged.
Build Images	Copies the `eventhub-env` secret to `.env` and runs `docker compose build`, producing six images tagged with the build number.	Build stops; no deployment. Typical causes: a Dockerfile error or a missing credential.
Deploy	Copies the secret again and runs `docker compose up -d --no-build --remove-orphans`: recreates containers whose image changed, removes containers of services no longer defined.	Build fails; some containers may already have been replaced, so the state is checked and, if needed, rolled back.
Smoke Test	`scripts/smoke-test.sh` requests five paths through the proxy: `/`, `/admin/`, `/api/health`, `/api/reports/health`, `/api/events`, retrying each up to 12 times, 5 s apart.	The new version is already live but the build turns red. Jenkins does not roll back by itself; the operator runs a rollback build (Section 10).

Stage	What it does	What happens on failure
post (always)	Reports the result. On success it prints the live version and sends a SUCCESS email. On failure it prints the rollback instructions and sends a FAILED email with a link to the console output. In both cases it deletes .env from the workspace.	The post section runs whether the build succeeds or fails, so the secret never stays on disk. If the email cannot be sent, the pipeline logs “Could not send email” and the build result does not change.

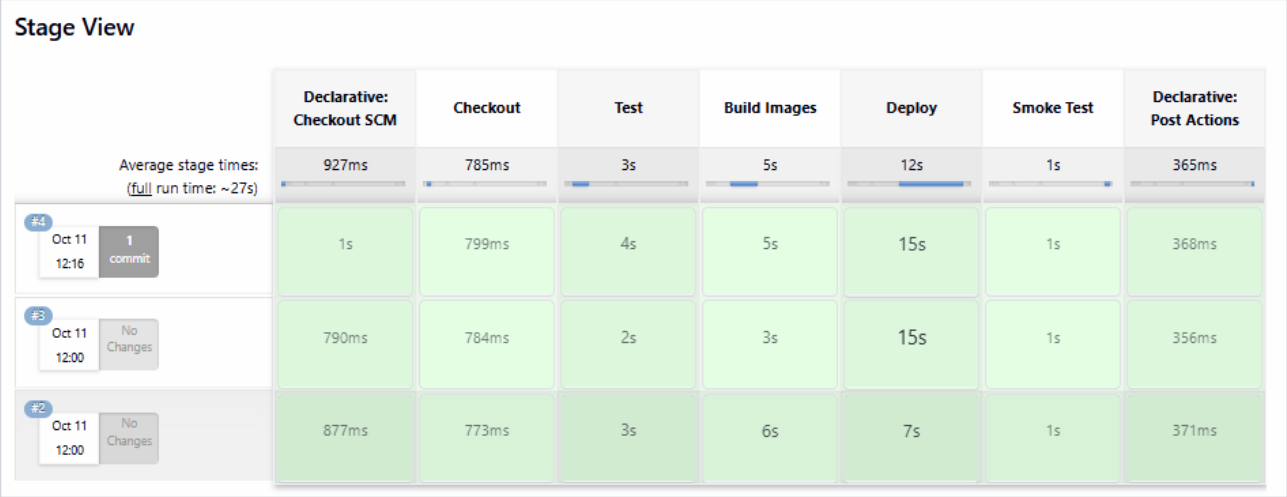


Figure 4. Jenkins Stage View, successful build. Build #4 started automatically from a push (polling trigger); builds #2 and #3 were started manually.

9. Testing Strategy

Testing has two layers. **Unit tests** check the logic of the two Node services before anything is deployed. **Smoke tests** check that the deployed system answers through the proxy and can reach its database. The unit tests were chosen because the APIs hold the business rules; the smoke test was chosen because the proxy, the static sites and the database have no unit-testable code, and it also proves the routing configuration works.

Module	Test	What it verifies
catalog-api	password hash verifies only the right password	scrypt hashing and verification
catalog-api	token round-trips and carries the role	Signed token contains the user's role
catalog-api	tampered, wrong-secret or expired tokens are rejected	Token signature and expiry checks
catalog-api	signup validation	Rejects invalid sign-up input
catalog-api	event validation normalizes the date and requires an image URL	Admin event input rules
catalog-api	registration form validation	Required registration fields
catalog-api	approval is rejected once the event is full	Capacity rule (`max_participants`)
report-service	fill rate is a rounded percentage	Fill-rate calculation
report-service	summary totals, full events, and most popular event	Report aggregation

How failures block deployment. The tests run in the `test` stage of each Dockerfile, started by the pipeline with `docker build --target test`. If `npm test` exits with a non-zero code, Docker aborts the build, the `sh` step fails, and Jenkins marks the stage and the build as failed. Because Build Images and Deploy come after Test, a failing test means no new image is built and the running version is not replaced. In the first Jenkins build, catalog-api passed 7 of 7 tests and report-service 2 of 2. These tests need no database or network, so they are fast and deterministic.

Smoke test gate. After deployment, a failure of any of the five checks fails the build, which flags the bad version immediately. The health endpoints run `SELECT 1`, so a green result also proves the API-to-database link.

Limits. There are no automated browser tests for the frontend and admin pages, and no integration tests that register users or approve registrations against a live database; those flows were checked manually.

10. Rollback Procedure and Data Persistence Proof

10.1 How rollback works

Every build tags all six images with its build number (for example `eventhub/proxy:3`), and older images stay on the Docker host. The Compose file selects images with `${TAG}`, so running the stack with a different `TAG` and `--no-build` switches to that older, already-built version in seconds, without rebuilding anything.

Procedure (Jenkins UI)

1. Open the `eventhub` job and choose **Build with Parameters**.
2. Enter the number of the last good build in `ROLLBACK_TAG` (for example `3`) and press **Build**.
3. The build skips Test and Build Images, copies the credential, runs `docker compose up -d --no-build` with `TAG=3`, then runs the smoke test.
4. Open `http://localhost:8090`: the footer shows `Build 3`, proving the old version is live.
5. Fix the code in Git (`git revert` of the bad commit, then push). Otherwise the next automatic build would deploy the bad code again.

Command-line alternative (PowerShell, from the project folder):

```
$env:TAG="3"
docker compose up -d --no-build
Remove-Item Env:TAG
```

10.2 Demonstration

A deliberate fault was introduced to prove the process: the hostname in the /api/ rule of proxy/nginx.conf was misspelled (catalog-apix instead of catalog-api) and merged through a pull request. Build #8 passed the Test, Build Images and Deploy stages but failed at the Smoke Test stage, and the site stopped working because the proxy container could not start. A rollback build (#9) with ROLLBACK_TAG = 7 restored the previous version, confirmed by the footer showing Build 7. The faulty commit was then reverted through a pull request, and build #10 passed.

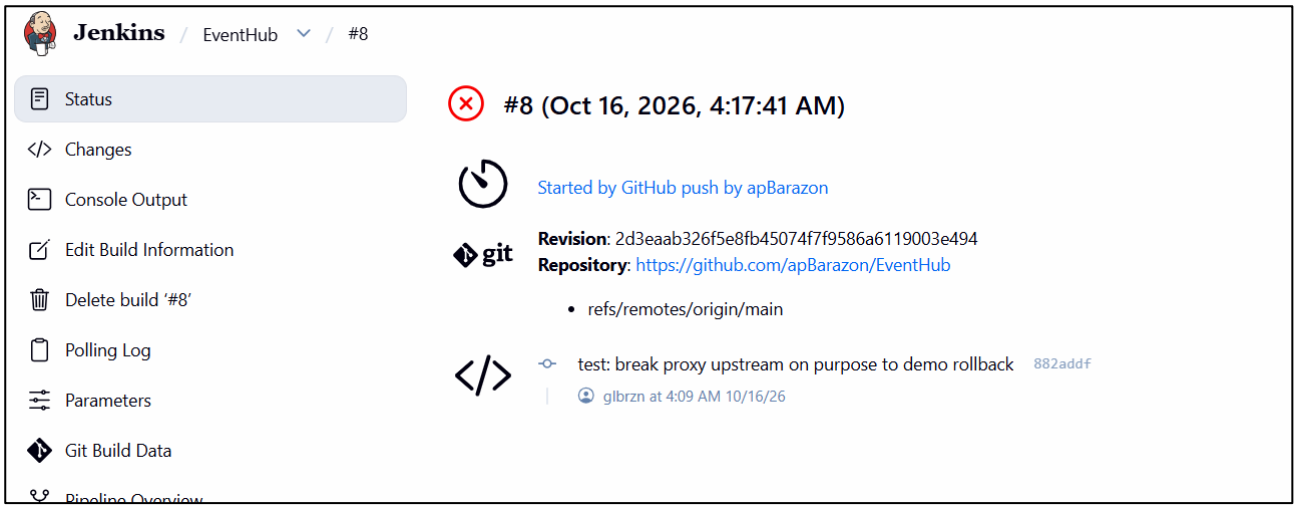


Figure 5. Failed build #8, started by a GitHub push (webhook) after the deliberately broken proxy change was merged to main.

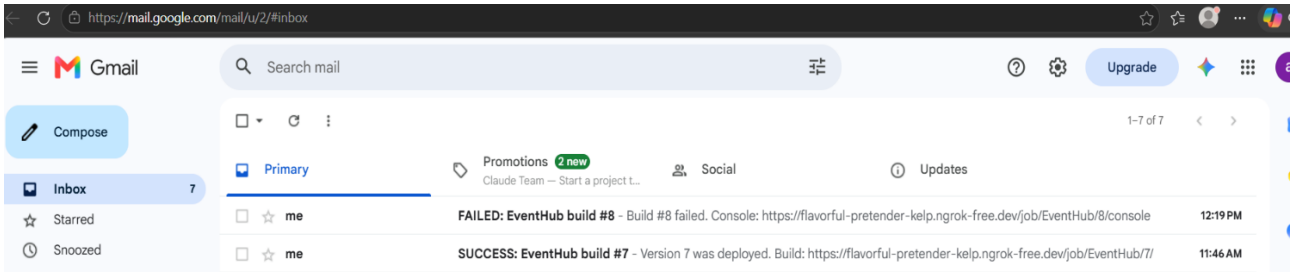


Figure 6. Email notifications from the post section: SUCCESS for build #7 and FAILED for build #8.

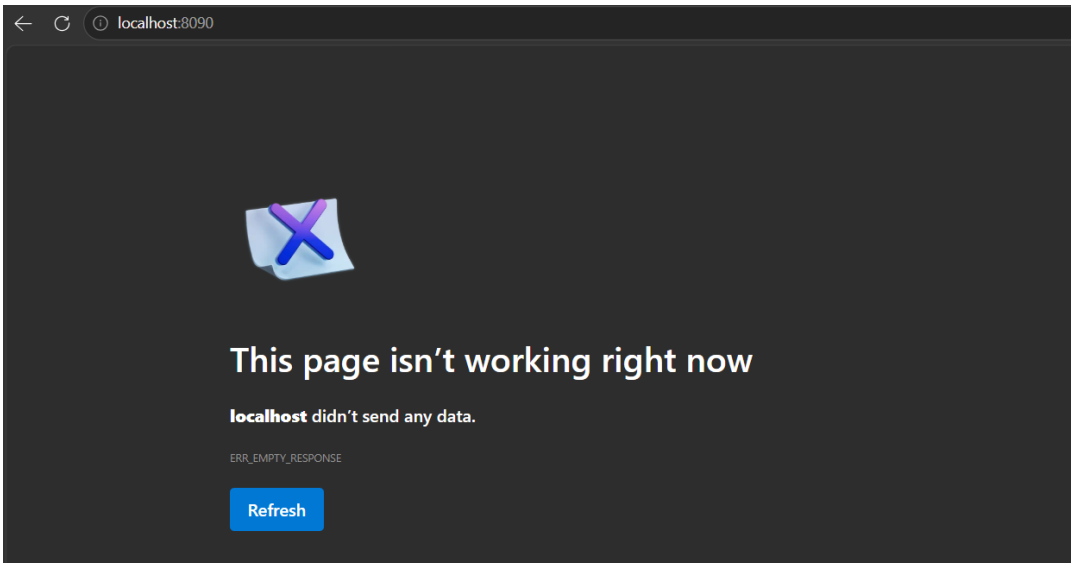


Figure 7. Browser error at http://localhost:8090 while the proxy container could not start, so the whole site was unreachable.

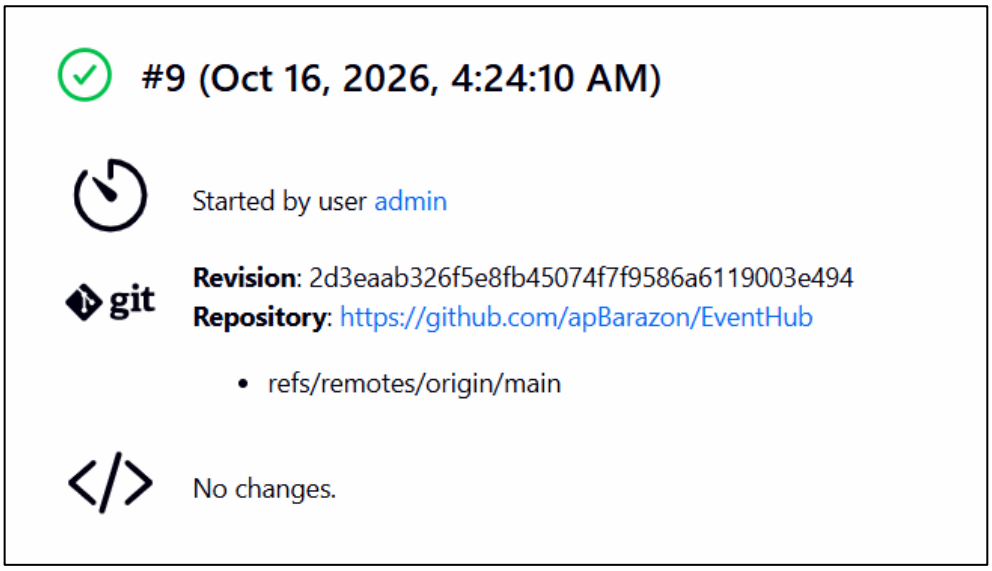


Figure 8. Rollback build #9, started manually from Build with Parameters with ROLLBACK_TAG = 7. It redeploys the images of build 7 without rebuilding.

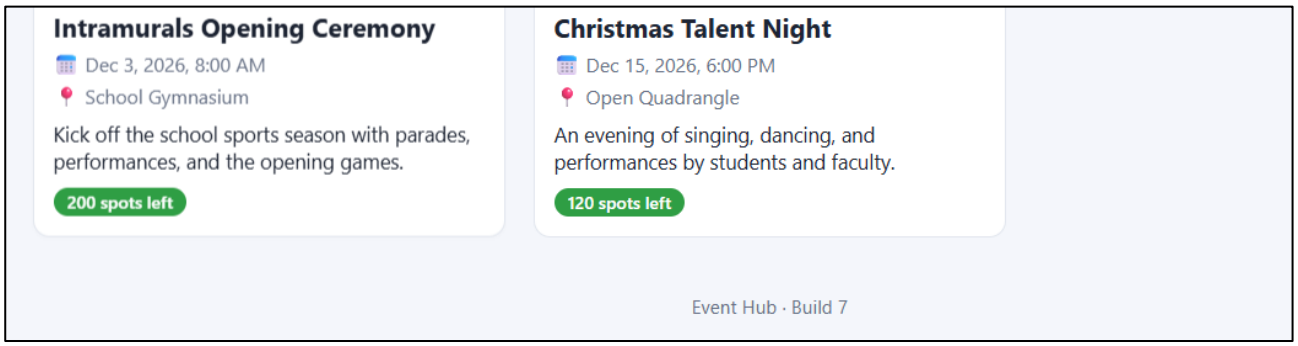


Figure 9. Page footer showing "Build 7" after the rollback, confirming that the previous version is live again.

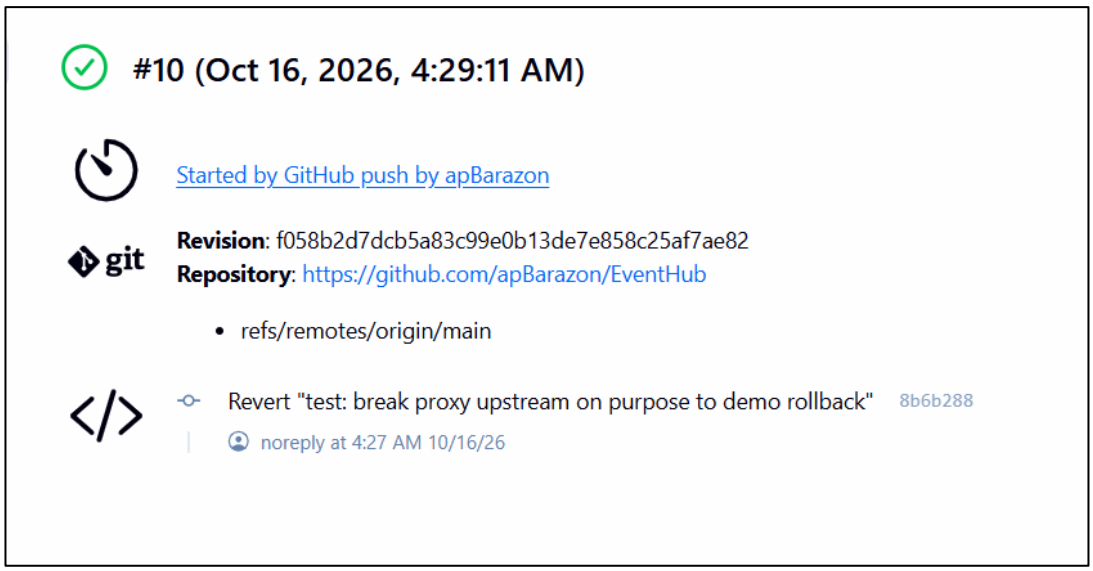


Figure 10. Build #10, started by a GitHub push after the faulty commit was reverted through a pull request. The pipeline passed again.

10.3 Data persistence proof

Application data lives in the named volume `db-data`, not in a container, so redeployments and rollbacks do not affect it. The proof follows these steps: (1) create data through the UI, for example a registration or a new event;

(2) run a rollback or redeploy build, which recreates every container; (3) query the database and show that the same rows are still present:

```
docker compose exec db mysql -u root -p eventhub -e "SELECT id, title FROM events; SELECT id, event_id, status FROM re
```

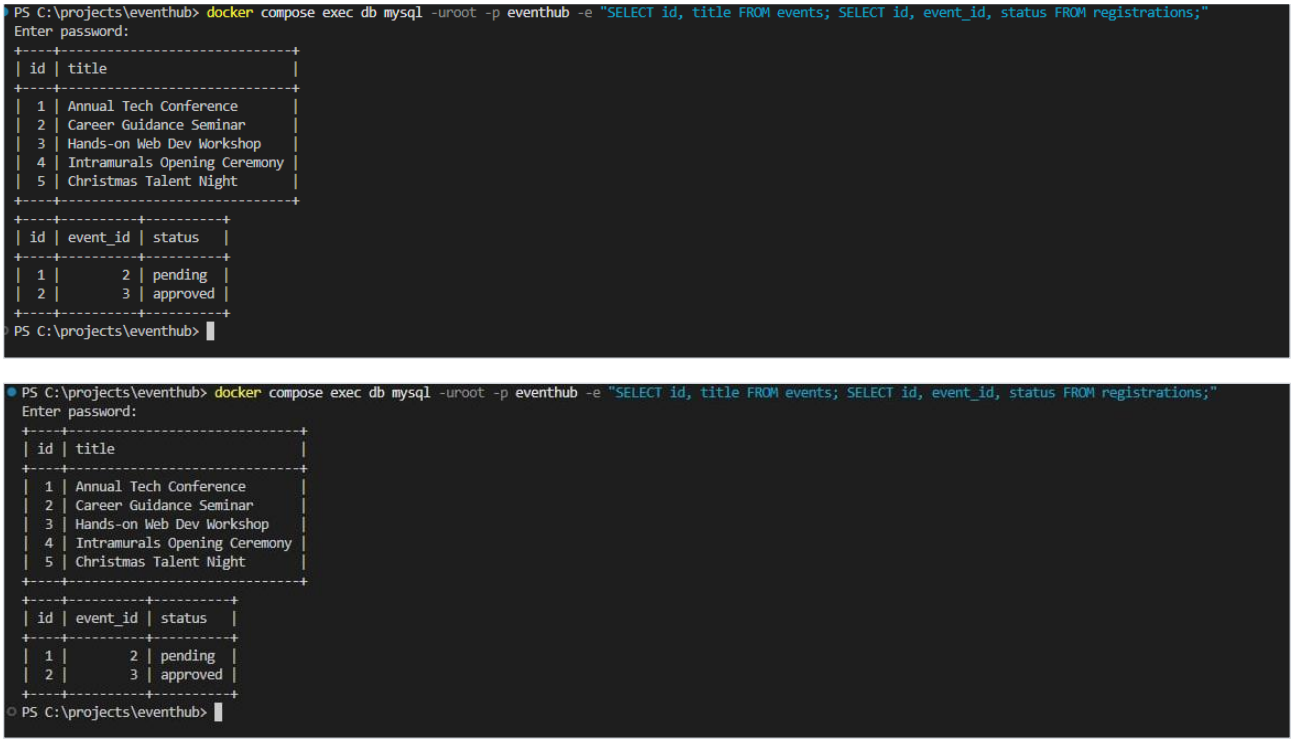


Figure 7. Data persistence proof: the same events and registrations before (top) and after (bottom) the failed deploy, the rollback and the redeploy.

11. Problems Encountered and Solutions

#	Problem and cause	Solution
1	**Jenkins web page did not load.** After starting the stack, the Jenkins page could not be opened in the browser. It looked like a port conflict, but docker ps -a showed that the Jenkins container had never been started.	Started it with docker compose up -d --build in infra/jenkins and waited one to two minutes for Jenkins to finish starting. Lesson: check the container state with docker ps -a before assuming a port conflict.
2	**Build #1 failed at Smoke Test and frontend/admin-panel showed unhealthy,** although the site worked. Alpine's `wget` resolved `localhost` to IPv6 while nginx listened on IPv4, so the checks were refused.	Replaced `localhost` with `127.0.0.1` in `scripts/smoke-test.sh` and in both static-site health checks, then pushed; the next build was green. Lesson: a change only takes effect after the image is rebuilt.
3	**Smoke Test stage could not run its script.** `scripts/smoke-test.sh` was committed from Windows without the executable permission (Git mode 100644). Jenkins runs `./scripts/smoke-test.sh` inside a Linux container, so the stage failed even though the stack had been deployed.	Set the executable bit in Git (mode 100644 to 100755) and pushed the fix as "fix(ci): make smoke test script executable" (Figure 8). The repository also has a `.gitattributes` rule (`*.sh text eol=lf`) so shell scripts keep Unix line endings on Windows. Lesson: Windows does not track file permissions, so Git has to be told explicitly.

#	Problem and cause	Solution
4	**Admin login failed after restarting the computer.** The API tried to create the admin before MySQL was ready, logged `Admin seed failed: ECONNREFUSED`, and does not retry. `depends_on` health ordering applies only to `compose up`, not to automatic restarts.	Restarted catalog-api once the database was healthy (`docker compose restart catalog-api`). Planned improvement: retry the seed until the database answers.
5	**404 for /admin/reports/summary.** That path is routed to the admin panel, which has no such file; the report API is under `/api/reports/`.	Used `/api/reports/summary` with a bearer token. The routing table in Section 3.3 now documents this.

12. Individual Contributions and References

12.1 Individual contributions

Roles decide who led each area of the work; every member is prepared to explain the whole system during the Q&A. The table lists what each member was responsible for, matched to the files in the repository.

Member	Role	What they did
Barazon, Angelo P.	Project Lead / Scrum Master; Backend and Database Engineer; DevOps and Infrastructure support	• Planned the tasks and timeline, ran the group meetings, managed the GitHub repository and put together the final demo flow. • Built `catalog-api` (authentication, events, registrations, full-event approval rule, notifications) and `report-service` (GET `/reports/summary`). • Wrote the Dockerfiles: multi-stage builds with a test stage for the APIs, two-stage builds for the static sites, and the proxy and db images. • Designed the database schema and seed data in `db/init.sql`. • Set up Jenkins in Docker (`infra/jenkins/Dockerfile` and `docker-compose.yml`, `jenkins_home` volume, Docker CLI and socket access). • Wrote the unit tests for both services (7 in catalog-api, 2 in report-service). • Supported the Jenkins pipeline and the Compose setup.
Laxamana, Karl Kristian Kenneth N.	DevOps / CI-CD Engineer	• Installed the plugins and created the `eventhub-env` secret-file credential. • Configured the pipeline job (Pipeline script from SCM) and the build triggers. • Wrote and maintains the `Jenkinsfile`: stages, build-number tags, the ROLLBACK_TAG parameter and the post section.
Villanueva, Martin M.	Infrastructure Engineer	• Designed the architecture and drew the diagram (Figure 1). • Wrote `docker-compose.yml`: network `eventhub-net`, volume `db-data`, health checks, `depends_on` conditions and the TAG variable. • Configured the nginx reverse proxy (`proxy/nginx.conf`), `.dockerignore` files and `.env.example`.

Member	Role	What they did
Tanglao, Joshua Lawrance S.	Frontend, QA, and Documentation Lead	• Built the student site (`frontend`) and the admin site (`admin-panel`) in HTML, CSS and vanilla JavaScript, including the build number in the footer. • Wrote the smoke test (`scripts/smoke-test.sh`). • Maintains the README and this technical documentation, and prepared the presentation slides.

Appendix A: Project Proposal

Completed Project Proposal Template from the course guidelines (Appendix A), filled in with the final project data.

Item	Details
Group name	Barazon Group
Project title	EventHub: School Event Registration System
Short description	EventHub is a containerized web application for school event registration. Students browse events, sign up, register through a short form and receive in-app notifications. An administrator creates and edits events, approves or rejects registrations and reads a summary report. Six containers run behind one nginx reverse proxy, and a Jenkins pipeline builds, tests, deploys and verifies the stack every time code is pushed to GitHub.
Repository URL	https://github.com/apBarazon/EventHub
Project Lead / Scrum Master	Barazon, Angelo P.
DevOps / CI-CD Engineer	Laxamana, Karl Kristian Kenneth N.
Infrastructure Engineer	Villanueva, Martin M.
Backend and Database Engineer	Barazon, Angelo P.
Frontend, QA, and Documentation Lead	Tanglao, Joshua Lawrance S.

Module / Container	Technology	Purpose	Justification
proxy	nginx 1.27-alpine	Single public entry point; routes /, /admin/, /api/ and /api/reports/ (published on 8090).	Small and fast. Path-based routing gives one public port, and it is the only container that publishes a port.
frontend	HTML, CSS, vanilla JavaScript on nginx-unprivileged 1.27-alpine	Student site: events, sign-up and login, registration form, notifications, build number in the footer.	No build tool or framework needed. The team knows the plain web stack; the unprivileged image runs as non-root.
admin-panel	HTML, CSS, vanilla JavaScript on nginx-unprivileged 1.27-alpine	Admin site: dashboard and report, event management, approve or reject registrations.	Same stack as the frontend, kept as its own module so the admin interface is separate from the student site.
catalog-api	Node.js 20 (built-in http, mysql2)	Authentication, events, registrations, approval logic, notifications; GET /health.	One language across the services; a single npm dependency keeps images small; built-in node:test for unit tests.
report-service	Node.js 20	GET /reports/summary (admin only): totals, fill rate, full events, most popular event.	A separate reporting module that verifies the same signed token as catalog-api.
db	MySQL 8.4	Relational data (users, events, registrations, notifications) in the named volume db-data; port not published.	The only database taught in our course; relational constraints fit the data.

Pipeline Plan	Details
Git hosting	GitHub (public repository, so Jenkins needs no Git credential).
Trigger (webhook / polling)	Poll SCM every two minutes, `pollSCM('H/2 * * * *')`; a GitHub webhook trigger, `githubPush()`, is also declared in the Jenkinsfile.
Planned stages	Checkout, Test, Build Images, Deploy, Smoke Test, then a post section that removes `.env` and reports the result.
Tests to be run	Unit tests (node:test) for catalog-api and report-service, run with `docker build --target test`; smoke test of five paths through the proxy after deployment.
Planned bonus features (optional)	Parameterized rollback build (`ROLLBACK_TAG`); e-mail notification from the post section.